External data sources {#external-data-sources}

> [!BEGINSHADEBOX]

On this page: Connect to third-party REST APIs and configure authentication so you can pull external data into your journeys for conditions and personalization.

> [!ENDSHADEBOX]

> [!CONTEXTUALHELP] id="ajo_journey_data_source_custom" title="External data sources" abstract="External data sources allow you to define a connection to third-party systems, for example if you are using a hotel booking system to check if the person has registered a room. As opposed to the built-in Adobe Experience Platform data source, you can create as many external data sources as you need."

Work with external data sources {#gs-ext-data-sources}

External data sources allow you to define a connection to third-party systems, for example if you are using a hotel booking system to check if the person has registered a room. As opposed to the built-in [!DNL Adobe Experience Platform] data source, you can create as many external data sources as you need.

> [!NOTE]
>
> - Guardrails when working with external systems are listed on this page.
> - As the responses are now supported, you should use custom actions instead of data sources for external data sources use-cases. For more information on responses, see custom action responses. Custom Actions without Data Lake persistence are the right choice when the data is only useful inside the journey and the external system is accessible via an API endpoint. For a comparison of all data access options, see Choose your data access strategy.

REST APIs using POST or GET and returning JSON are supported. API Key, basic and custom authentication modes are supported.

Let's take the example of a weather API service that I want to use to customize my journey's behaviors according to real-time weather data.

Here are two examples of the API call:

- *https://api.adobeweather.org/weather?city=London,uk&appid=1234*
- *https://api.adobeweather.org/weather?lat=35&lon=139&appid=1234*

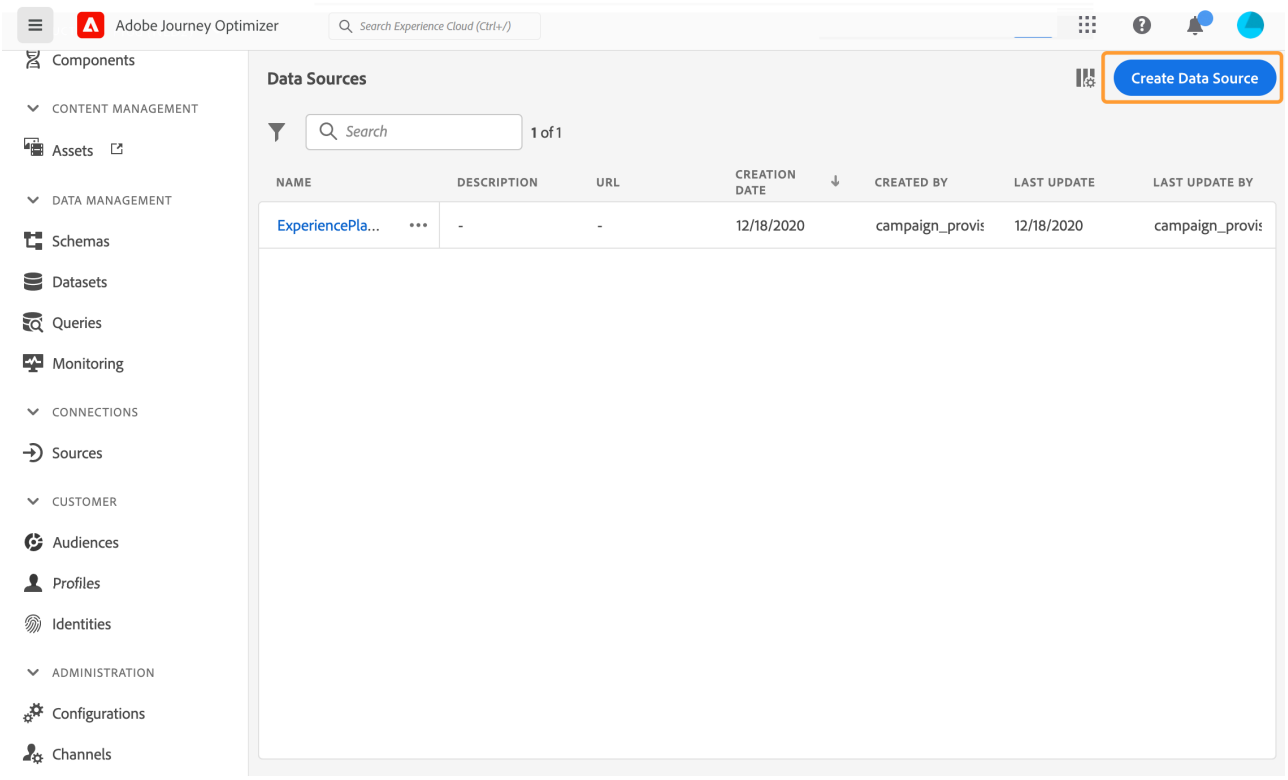
The call is composed of a main URL (*https://api.adobeweather.org/weather*), two parameter sets ("city" for the city and "lat/long" for the latitude and longitude) and the API key (appid).

> [!TIP]
>
> We recommend leaving at least a one-minute buffer between the external API's token expiration period and your Journey Optimizer `cacheDuration` setting, especially under heavy workloads, to avoid expiration mismatches and 401 errors.

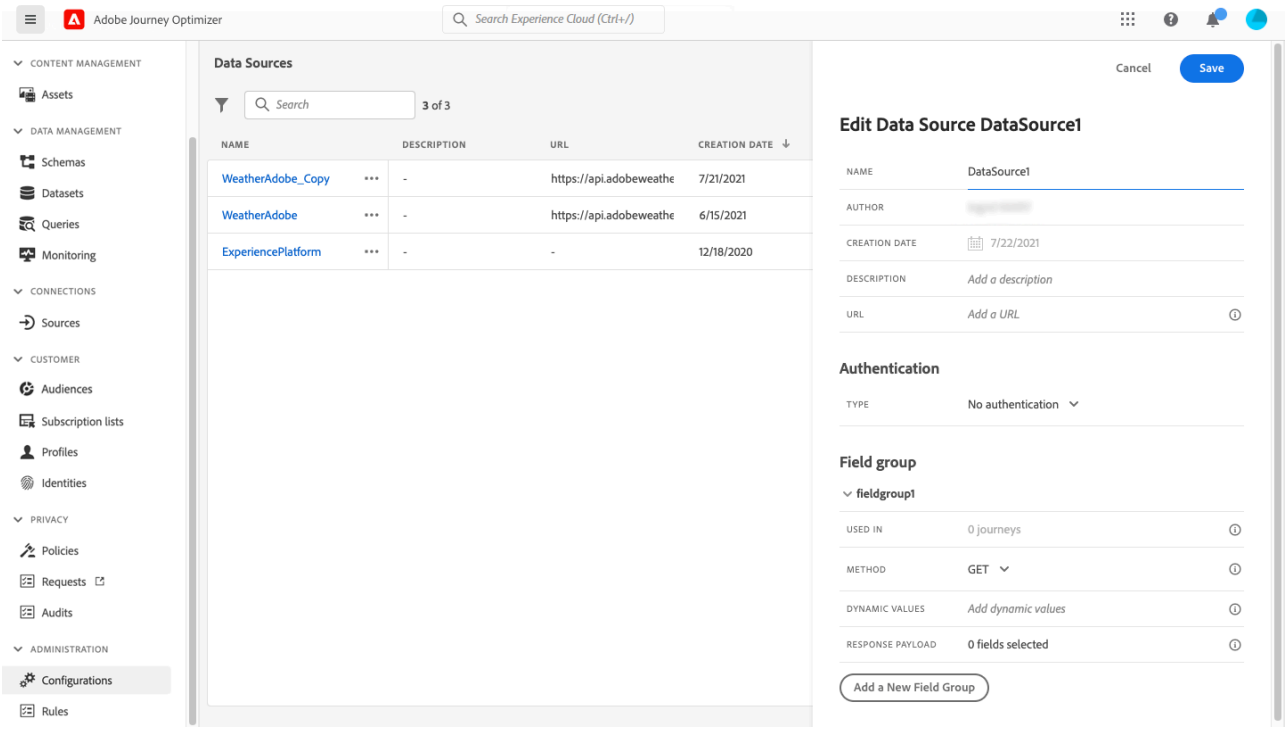
Create and configure an external data source {#create-ext-data-sources}

Below are the main steps to create and configure a new external data source:

1. From the list of data sources, click **[!UICONTROL Create Data Source]** to create a new external data source.



This opens the data source configuration pane on the right-hand side of the screen.



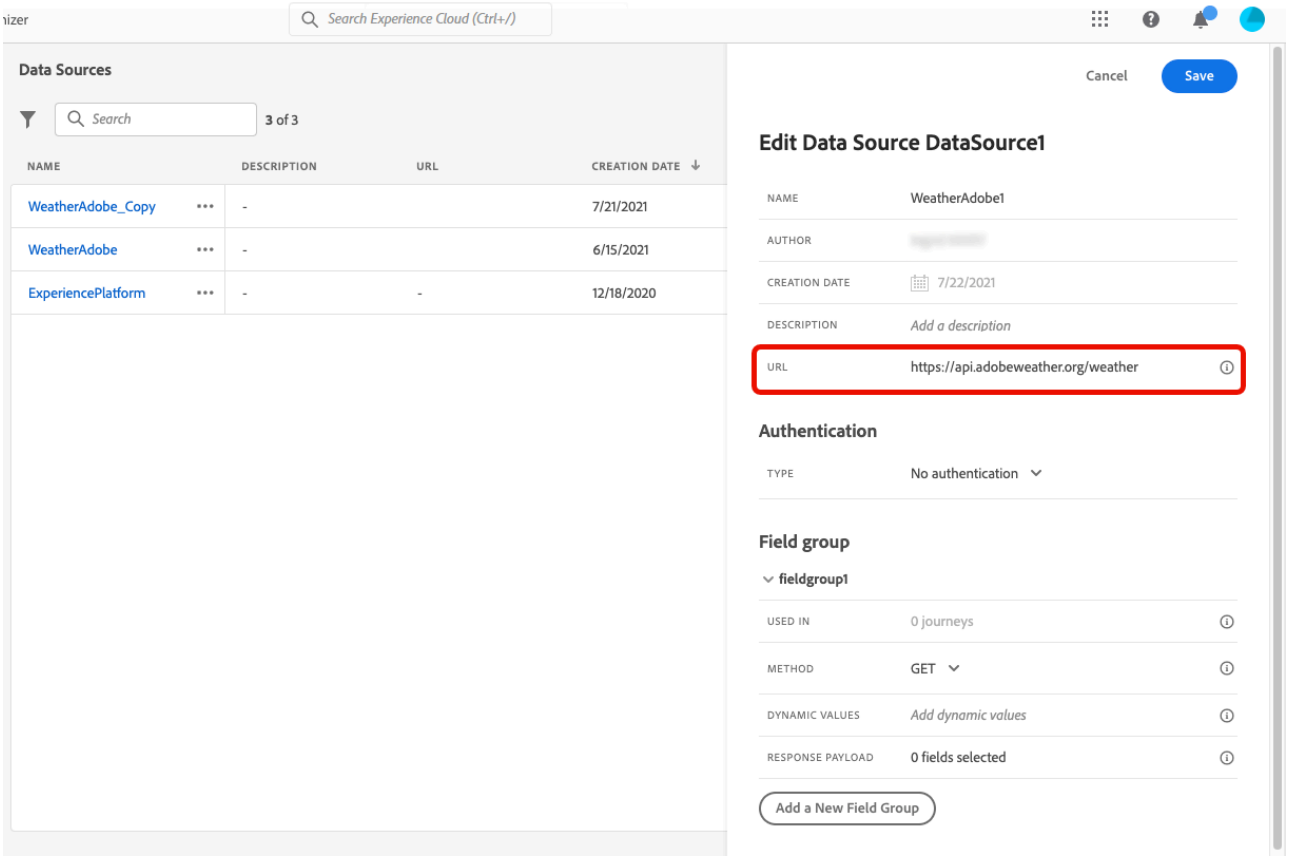
2. Enter a name for your data source.

Only alphanumeric characters and underscores are allowed. The maximum length is 30 characters.

1. Add a description to your data source. This step is optional.

2. Add the URL of the external service. In our example: *https://api.adobeweather.org/weather*.

> [!CAUTION]
>
> We strongly recommend using HTTPS for security reasons. Also note that we do not allow the use of Adobe addresses that are not publicly available and the use of IP addresses.



3. Configure the authentication depending on the external service configuration: **[!UICONTROL No authentication]**, **[!UICONTROL Basic]**, **[!UICONTROL Custom]** or **[!UICONTROL API key]**.

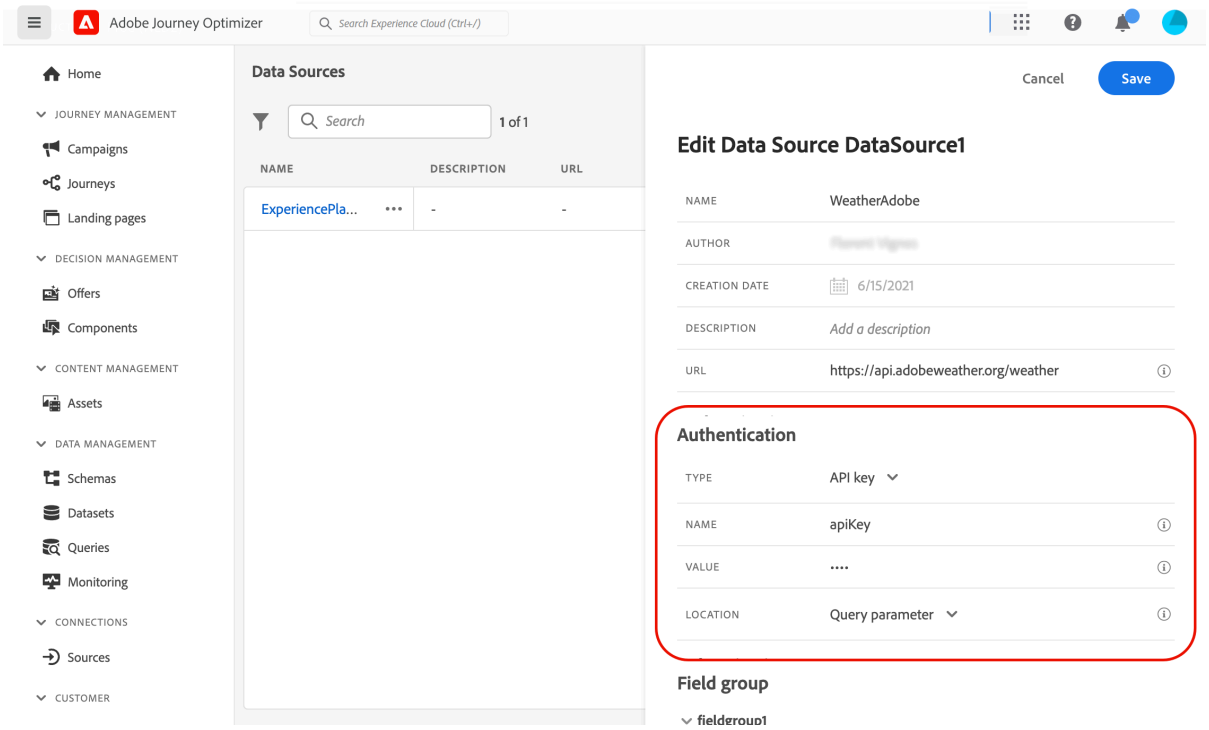
   For the basic authentication mode, you need to fill in a username and a password.

> [!NOTE]
>
> - When the authentication call is performed, the `<username>:<password>` string, encoded in base64, is added in the Authentication header.
> - [!DNL Adobe Journey Optimizer] automatically encrypts secrets defined in custom actions. Each organization's encryption keys are securely managed in a dedicated vault tied to their organization. When credentials are displayed in the interface, they are masked by default to prevent accidental exposure.

   For more information about the custom authentication mode, see the custom authentication mode section. In our example, we choose the API key authentication mode, as below:

   - **[!UICONTROL Type]**: "API key"
   - **[!UICONTROL Name]**: "appid" (this is the API key parameter name)

- **[!UICONTROL Value]**: "1234" (this is the value of our API key)
- **[!UICONTROL Location]**: "Query parameter" (the API key is located in the URL)



4. Add a new field group for each API parameter set by clicking **[!UICONTROL Add a New Field Group]**. Only alphanumeric characters and underscores are allowed in the field group name. The maximum length is 30 characters. In our example, we need to create two field groups, one for each parameter set (city and long/lat).

For the "long/lat" parameter set, we create a field group with the following information:

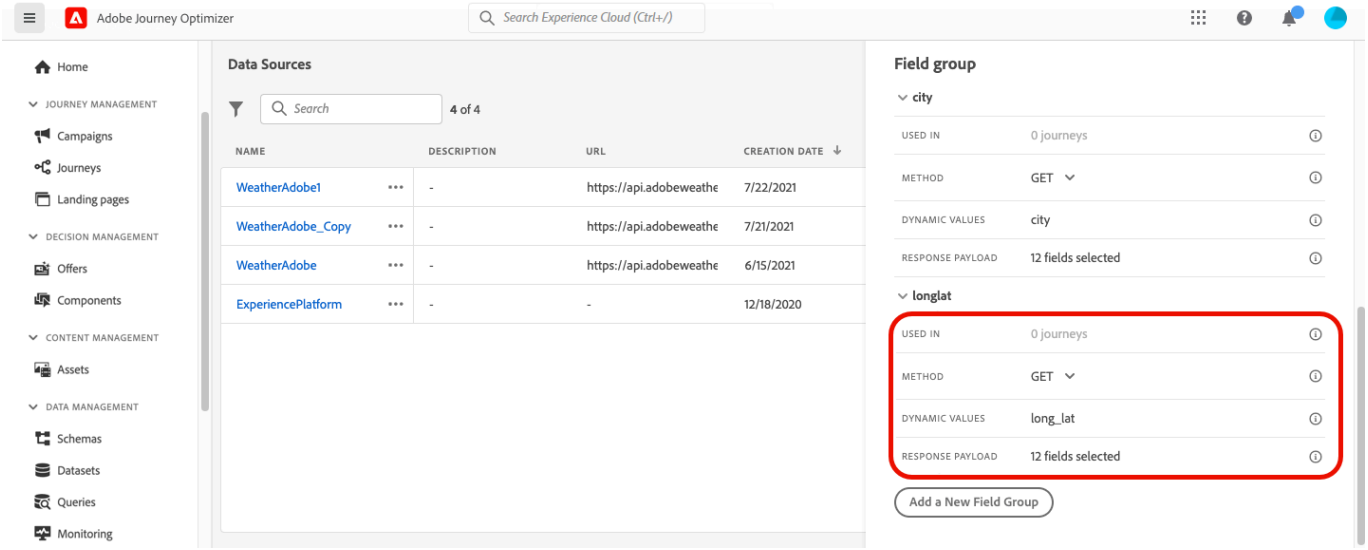
- **[!UICONTROL Used in]**: displays the number of journeys that use a field group. You can click the **[!UICONTROL View journeys]** icon to display the list of journeys using this field group.
- **[!UICONTROL Method]**: select the POST or GET method. In our case, we select the GET method.
- **[!UICONTROL Dynamic Values]**: enter the different parameters separated by a comma, "long,lat" in our example. Since the parameter values depend on the execution context, they will be defined in the journeys. Learn more about expressions
- **[!UICONTROL Response Payload]**: click inside the **[!UICONTROL Payload]** field and paste an example of the payload returned by the call. For our example, we used a payload found on a weather API website. Verify that the field types are correct. Each time the API is called, the system will retrieve all the fields included in the payload example. Note that you can click on **[!UICONTROL Paste a new payload]** if you want to change the payload currently passed.
- **[!UICONTROL Sent Payload]**: this field does not appear in our example. It is only available if you select the POST method. Paste the payload that will be sent to the third-party system.

In case of a GET call requiring parameter(s), you enter the parameter(s) in the **[!UICONTROL Dynamic Values]** field and they are automatically added at the end of the call. In case of a POST call, you need to:

- list the parameters to be passed at call time in the **[!UICONTROL Dynamic Values]** field (in the example below: "identifier").

- specify them also with the exact same syntax in the body of the sent payload. To do so, you need to add: "param": "name of your parameter" (in the example below: "identifier"). Follow the syntax below:

```
{"id":{"param":"identifier"}}
```



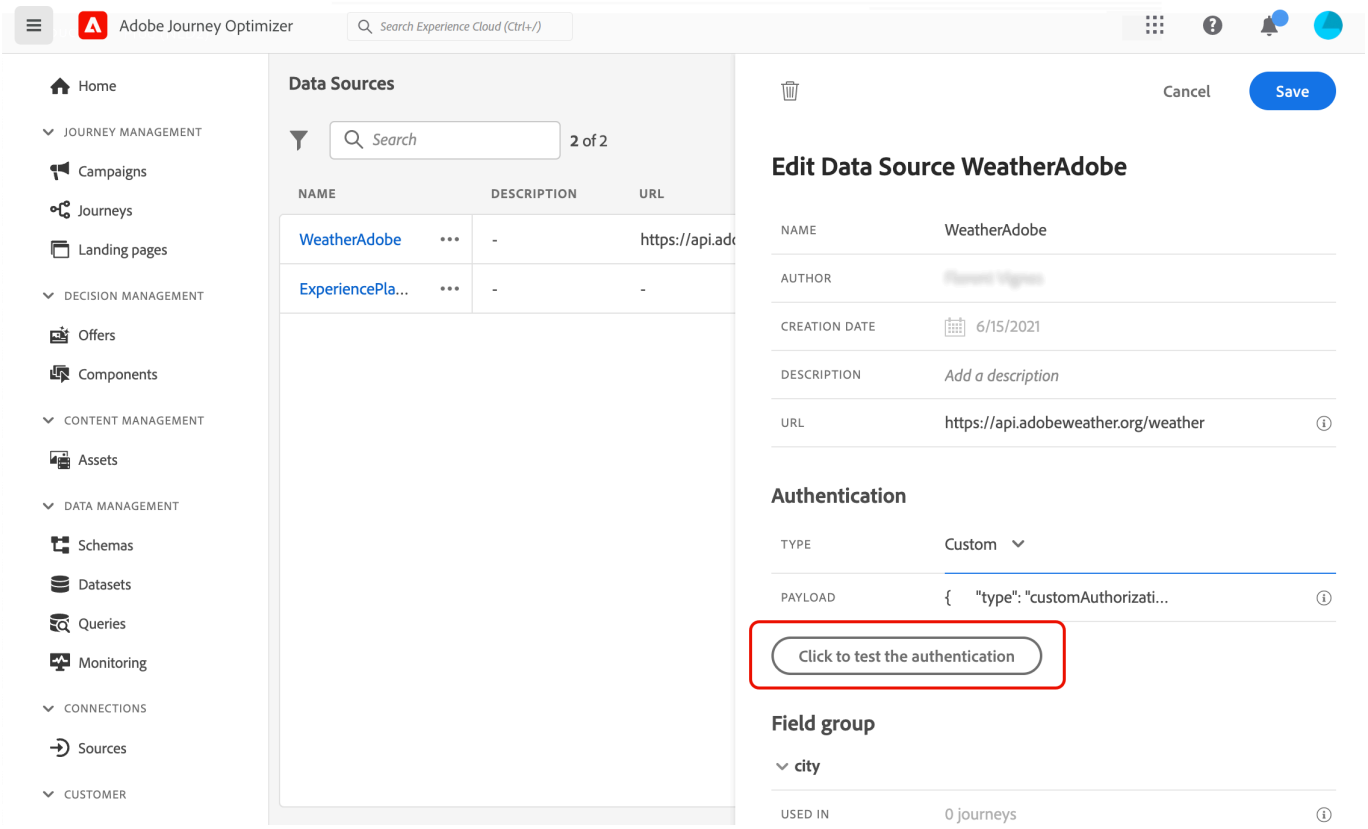
Once your changes are saved, the data source is configured and ready to be used in your journeys, for example in your conditions or to personalize an email. If the temperature is above 30°C, you can decide to send a specific communication.

Custom authentication mode {#custom-authentication-mode}

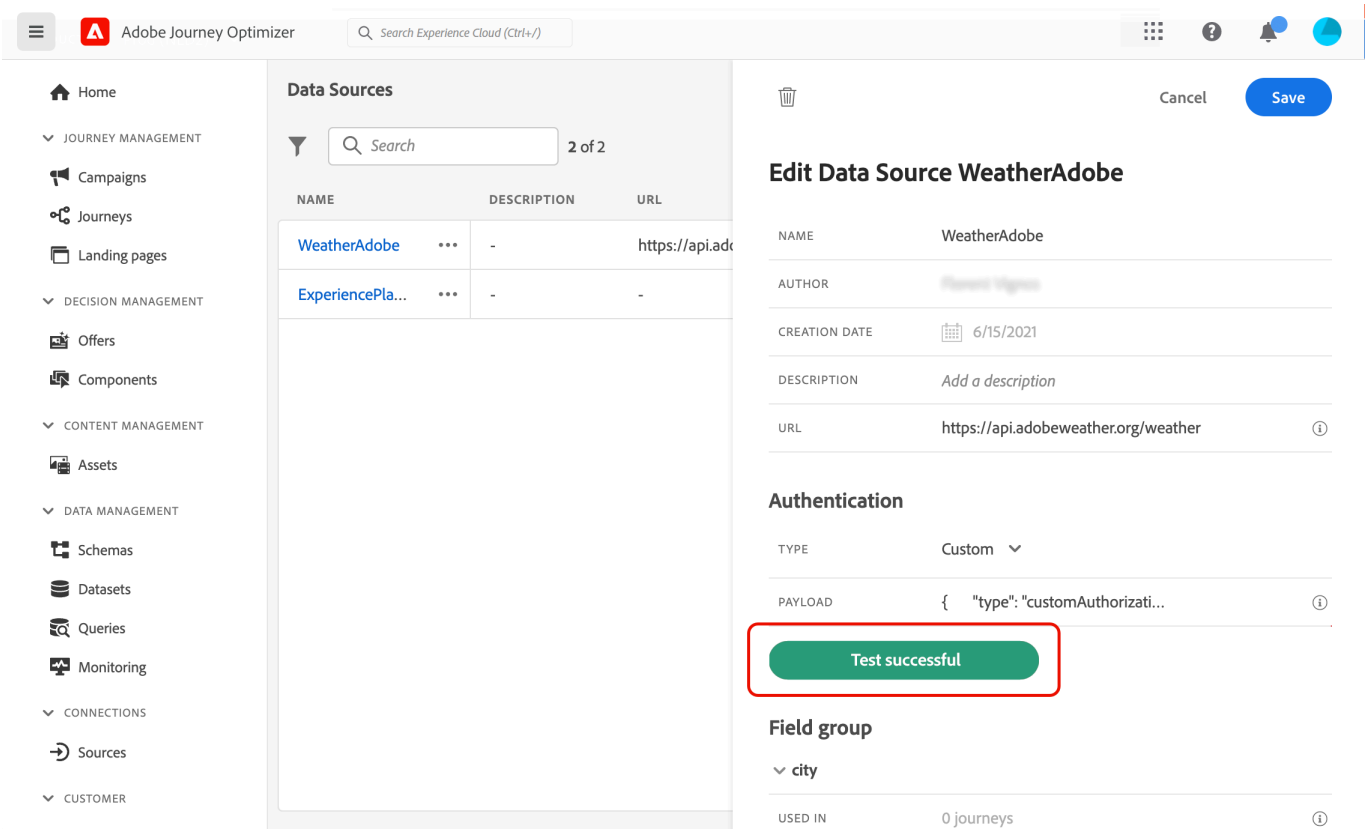
> [!CONTEXTUALHELP] id="jo_authentication_payload" title="About custom authentication" abstract="The custom authentication mode is used for complex authentication to call API wrapping protocols such as OAuth2. The action execution is a two-step process. First, a call to the endpoint is performed to generate the access token. Then, the access token is injected in the HTTP request of the action."

The custom authentication mode is used for complex authentication, frequently used to call API wrapping protocols such as OAuth2, to retrieve an access token to be injected in the real HTTP request for the action.

When you configure the custom authentication, use the **[!UICONTROL Click to check the authentication]** button to control if the custom authentication payload is correctly configured.



When the test is successful, the button turns green.



With this authentication mode, the action execution is a two-step process:

1. Call the endpoint to generate the access token.
2. Call the REST API by injecting in the proper way the access token.

> [!NOTE]

> **This authentication has two parts.**

Definition of the endpoint to be called to generate the access token{#custom-authentication-endpoint}

- `endpoint`: URL to use to generate the endpoint
- method of the HTTP request on the endpoint (`GET` or `POST`)
- `headers`: key-value pairs to be injected as headers in this call if required
- `body`: describes the body of the call if the method is POST. We support a limited body structure, defined in the bodyParams (key-value pairs). The bodyType describes the format and encoding of the body in the call:
  - `form`: meaning that the content type will be application/x-www-form-urlencoded (charset UTF-8) and the key-value pairs will be serialized as is: key1=value1&key2=value2&...
  - `json`: meaning that the content type will be application/json (charset UTF-8) and the key-value pairs will be serialized as a json object as is: *{ "key1": "value1", "key2": "value2", ...}*

Definition of the way the access token must be injected in the HTTP request of the action{#custom-authentication-access-token}

- **authorizationType**: defines how the generated access token must be injected in the HTTP call for the action. The possible values are:
  - `bearer`: indicates that the access token must be injected in the Authorization header, such as: *Authorization: Bearer <access token>*
  - `header`: indicates that the access token must be injected as a header, the header name defined by the property `tokenTarget`. For instance, if the `tokenTarget` is `myHeader`, the access token will be injected as a header as: *myHeader: <access token>*
  - `queryParam`: indicates that the access token must be injected as a queryParam, the query param name defined by the property tokenTarget. For instance, if the tokenTarget is myQueryParam, the URL of the action call will be: *<url>?myQueryParam=<access token>*
- **tokenInResponse**: indicates how to extract the access token from the authentication call. This property can be:
  - `response`: indicates that the HTTP response is the access token
  - a selector in a json (assuming that the response is a json, we do not support other formats such as XML). The format of this selector is *json://<path to the access token property>*. For instance, if the response of the call is: *{ "access_token": "theToken", "timestamp": 12323445656 }*, the tokenInResponse will be: *json: //access_token*

The format of this authentication is:

```
{
    "type": "customAuthorization",
    "endpoint": "<URL of the authentication endpoint>",
    "method": "<HTTP method to call the authentication endpoint, in 'GET' or
'POST'>",
    (optional) "headers": {
        "<header name>": "<header value>",
```

```
        ...
    },
    (optional, mandatory if method is 'POST') "body": {
        "bodyType": "<'form'or 'json'>,
        "bodyParams": {
            "param1": value1,
            ...
        }
    },
    "tokenInResponse": "<'response' or json selector in format 'json://<field path
to access token>'",
    "cacheDuration": {
        (optional, mutually exclusive with 'duration') "expiryInResponse": "<json
selector in format 'json://<field path to expiry>'",
        (optional, mutually exclusive with 'expiryInResponse') "duration":
<integer value>,
        "timeUnit": "<unit in 'milliseconds', 'seconds', 'minutes', 'hours',
'days', 'months', 'years'>"
    },
    "authorizationType": "<value in 'bearer', 'header' or 'queryParam'>",
    (optional, mandatory if authorizationType is 'header' or 'queryParam')
"tokenTarget": "<name of the header or queryParam if the authorizationType is
'header' or 'queryParam'>",
}
```

> [!NOTE]
>
> Encode64 is the only function available in the authentication payload.

You can change the cache duration of the token for a custom authentication data source. Below is an example of a custom authentication payload. The cache duration is defined in the `cacheDuration` parameter. It specifies the retention duration of the generated token in the cache. The unit can be milliseconds, seconds, minutes, hours, days, months, years.

Here is an example for the bearer authentication type:

```
{
    "type": "customAuthorization",
    "endpoint": "https://<your_auth_endpoint>/epsilon/oauth2/access_token",
    "method": "POST",
    "headers": {
      "Authorization": "Basic EncodeBase64(<epsilon Client Id>:<epsilon Client
Secret>)"
    },
    "body": {
      "bodyType": "form",
      "bodyParams": {
        "scope": "cn mail givenname uid employeeNumber",
        "grant_type": "password",
        "username": "<epsilon User Name>",
        "password": "<epsilon User Password>"
```

```
      }
    },
    "tokenInResponse": "json://access_token",
    "cacheDuration": {
      "duration": 5,
      "timeUnit": "minutes"
    },
  },
```

> [!NOTE]
>
> - The authentication token is cached per journey: if two journeys are using the same custom action, each journey has its own token cached. That token is not shared between those journeys.
> - Cache duration helps to avoid too many calls to the authentication endpoints. Authentication token retention is cached in services, there is no persistence. If a service is restarted, it starts with a clean cache. The cache duration by default is 1 hour. In the custom authentication payload, it can be adapted by specifying another retention duration.

Certificate-based custom authentication {#certificate-credential}

For enterprise APIs that enforce certificate-based identity verification, such as Microsoft Entra ID, you can configure certificate-based custom authentication by adding `"subType": "certificateCredential"` to your custom authorization payload. By default, Journey Optimizer uses Adobe's managed certificate to sign a JWT client assertion with `RS256` and exchange it for an access token. No client secret is required for this default RS256 option. The same payload also supports HMAC signing with your OAuth client secret instead of an Adobe certificate.

This option adds two mandatory fields to the standard `customAuthorization` schema: `subType` and `aud`. All other fields (`endpoint`, `method`, body params, `tokenInResponse`) remain unchanged. When `subType` is absent, behavior is identical to standard custom authentication — existing configurations are not affected.

- **`subType`**: Set to `"certificateCredential"` to activate certificate-based authentication.
- **`aud`**: The audience value included in the JWT client assertion. For Microsoft Entra ID, this is the same as the `endpoint` URL, but it must always be explicitly set.
- **`alg`** (optional): The JWT signing algorithm. Supported values are `RS256` (default), `HS256`, `HS384`, and `HS512`. When `alg` is absent or set to `RS256`, the existing RSA flow runs unchanged. Any other value is rejected. Configure this field in the custom authorization payload; there is no separate UI control for the signing algorithm.

The `client_assertion` and `client_assertion_type` fields are never authored by the user. They are automatically injected by the platform at runtime, immediately before the token endpoint call.

How RS256 authentication works {#certificate-credential-how-it-works}

Certificate-based custom authentication implements OAuth 2.0 client credentials with a JWT client assertion, as defined in RFC 7523{target="_blank"} — the same standard supported by Microsoft Entra ID and Okta. Instead of a client secret, Journey Optimizer proves its identity using a JWT signed with Adobe's managed

private key. Your Identity Provider verifies the signature using Adobe's public certificate, which you register once in your Identity Provider.

The token exchange follows these steps:

1. Journey Optimizer builds a JWT client assertion signed with Adobe's private key.
2. The assertion is sent to your token endpoint alongside your `client_id`, `grant_type`, and `scope`.
3. Your Identity Provider verifies the JWT signature against Adobe's registered public certificate.
4. Your Identity Provider returns a Bearer access token.
5. Journey Optimizer uses that token to call your custom action endpoint.

Adobe certificate details {#certificate-credential-details}

For the default RS256 option, Adobe manages the certificate and its associated private key. The following table summarizes its key properties. These certificate requirements do not apply to HMAC signing.

Property	Value
Issued by	DigiCert (public CA)
Managed by	Adobe
Algorithm	RS256 (RSA), the default JWT signing option
What to register in your Identity Provider	Adobe's leaf certificate only — not the intermediate or root CA
How to obtain	Retrieve it from the mTLS Public Certificate API{target="_blank"} (see the **Certificate** guardrail below)
Rotation	Adobe automatically rotates the certificate 60 days before expiry (certificate lifetime: 13 months). The previous certificate remains valid until 30 days before expiry. Customers are not currently notified of rotation — periodically call the mTLS Public Certificate API{target="_blank"} to check the `expiryDate` and re-configure your IDP before the old certificate is revoked.

Adobe automatically rotates the certificate 60 days before expiry. The previous certificate remains valid until 30 days before expiry. Customers are not currently notified — see the **Certificate (RS256 only)** guardrail below for how to monitor rotation programmatically.

JWT assertion structure {#certificate-credential-jwt}

> [!BEGINSHADEBOX]

Keep the following guardrails in mind when configuring certificate-based custom authentication:

- **Token endpoint URL**: Must be HTTPS. Avoid URLs containing `?`, this is a sign the authorization endpoint was pasted instead of the token endpoint.
- `method`: Must be `POST`. OAuth token endpoints only accept POST requests.

- `client_id`: Must not be blank and must have no leading or trailing whitespace. A blank value produces a valid-looking JWT that the Identity Provider will reject with an opaque error.
- `scope`: Expressed as a single space-separated string in `bodyParams`. Maximum 1000 characters total.
- **Certificate (RS256 only)**: Adobe manages the certificate and private key; you never upload or enter a certificate. Before using the custom action in a live journey with RS256, you must register **Adobe's leaf certificate** in your Identity Provider. To retrieve it, call the mTLS Public Certificate API{target="_blank"} and look for the entry where `certCommonName` is `ajo-journeys.aep-mtls.adobe.com`. Register the `publicCertificate` value from that entry, not the intermediate or root CA certificates. Since customers are not currently notified of certificate rotation, you must periodically call the mTLS Public Certificate API to check the `expiryDate` and update the registered certificate in your IDP before the old certificate is revoked 30 days before expiry. HMAC signing does not require Adobe certificate registration.

> [!ENDSHADEBOX]

You do not author the JWT client assertion, Journey Optimizer generates and signs it for you. The expected structure is provided here so your Identity Provider team can validate the claims.

The following header and payload apply to the default RS256 option. For HMAC differences, see HMAC-signed JWT client assertions.

Header:

```
{
  "alg": "RS256",
  "x5t": "<base64url SHA-1 thumbprint of Adobe's leaf certificate>"
}
```

Payload:

```
{
  "iss": "<client_id>",
  "sub": "<client_id>",
  "aud": "<token endpoint URL>",
  "iat": "<current unix timestamp>",
  "exp": "<iat + 600 seconds>",
  "jti": "<unique UUID per request>"
}
```

Note the following:

- For RS256, `exp` is `iat` + 600 seconds (10 minutes), consistent with Okta and Entra ID requirements. For HMAC, `exp` is `iat` + 300 seconds (5 minutes).
- Each assertion uses a unique `jti`, which makes it replay-attack safe.
- `client_assertion` and `client_assertion_type` are injected by the platform automatically and are never authored.

Here is an example for the certificate credential authentication type, for Microsoft Entra ID:

```
{
  "type": "customAuthorization",
  "subType": "certificateCredential",
  "aud": "https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/token",
  "authorizationType": "Bearer",
  "endpoint": "https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/token",
  "method": "POST",
  "body": {
    "bodyType": "form",
    "bodyParams": {
      "client_id": "<your-client-id>",
      "grant_type": "client_credentials",
      "scope": "https://api.example.com/.default"
    }
  },
  "tokenInResponse": "json://access_token"
}
```

Here is an example for the same certificate credential authentication type, for Okta:

```
{
  "type": "customAuthorization",
  "subType": "certificateCredential",
  "authorizationType": "bearer",
  "endpoint": "https://<your-okta-domain>/oauth2/v1/token",
  "aud": "https://<your-okta-domain>/oauth2/v1/token",
  "method": "POST",
  "body": {
    "bodyType": "form",
    "bodyParams": {
      "client_id": "<your-okta-app-client-id>",
      "grant_type": "client_credentials",
      "scope": "<your-api-scope>"
    }
  },
  "tokenInResponse": "json://access_token"
}
```

Here is an example for the header authentication type:

```
{
  "type": "customAuthorization",
  "endpoint": "https://myapidomain.com/v2/user/login",
  "method": "POST",
  "headers": {
    "x-retailer": "any value"
```

```
    },
    "body": {
      "bodyType": "form",
      "bodyParams": {
        "secret": "any value",
        "username": "any value"
      }
    },
    "tokenInResponse": "json://token",
    "cacheDuration": {
      "expiryInResponse": "json://expiryDuration",
      "timeUnit": "minutes"
    },
    "authorizationType": "header",
    "tokenTarget": "x-auth-token"
  }
```

Here is an example of the response of the login API call:

```
  {
    "token": "xDIUssuYE9beucIE_TFOmpdheTqwzzISNKeysjeODSHUibdzN87S",
    "expiryDuration" : 5
  }
```

> [!CAUTION]
>
> When configuring custom authentication for a custom action, note that nested JSON objects (e.g., sub-objects within `bodyParams`) are **supported**.

HMAC-signed JWT client assertions {#hmac-credential}

> [!AVAILABILITY]
>
> HMAC-signed JWT authentication is available in Limited Availability.

To use HMAC signing, set `alg` to `HS256`, `HS384`, or `HS512` in the custom authorization payload. Journey Optimizer uses your OAuth `client_secret` as the signing key. Your identity provider must accept `client_secret_jwt` with the same secret and algorithm. No Adobe certificate registration is required.

HMAC requires the same fields as RS256: `"subType": "certificateCredential"`, an explicit `aud`, `"method": "POST"`, `"bodyType": "form"`, and `client_id` in `body.bodyParams`. In addition, add `client_secret` to `body.bodyParams`. The minimum secret lengths are measured in UTF-8 bytes, not characters:

Algorithm	**Minimum `client_secret` length**
HS256	32 bytes
HS384	48 bytes

Algorithm	Minimum `client_secret` length
HS512	64 bytes

Journey Optimizer does **not** send `client_secret` to the token endpoint. It sends the other authored `bodyParams` and automatically adds `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer` and `client_assertion`.

The HMAC JWT uses `typ: JWT` and the selected `alg`, without `x5t`; `aud` is a single-element array and `exp` is `iat` + 300 seconds, compared with 600 seconds for RS256.

{{$include /help/_includes/do-not-localize/datasource/ai-augmented-external-data-sources.md}}